Neural Processing Letters (2026) 58:5
https://doi.org/10.1007/s11063-025-11819-w

Optimized Task Scheduling in Fog-Cloud Computing Using Hybrid Deep Learning and Metaheuristic Algorithms

Amir Heirati[1] · Mehdi Fartash[1] · Mohammad Khalily-Dermany[2]

Received: 15 February 2025 / Accepted: 17 November 2025 / Published online: 1 December 2025
© The Author(s) 2025

Abstract

Fog computing has emerged as an effective paradigm to address the latency issues inherent in cloud computing by enabling data processing closer to data sources, particularly for IoT applications in healthcare, smart transportation, and industrial automation. However, efficient task scheduling in fog environments remains challenging due to their dynamic, heterogeneous nature. This paper presents an integrated solution for task scheduling in fog-cloud environments to reduce energy consumption and minimize task makespan, while maintaining high-quality service (QoS). The proposed model comprises three core components: workload prediction, task offloading, and scheduling optimization. First, a Feedforward Neural Network (FNN) is used to predict workload, allowing for proactive resource allocation. Next, a reinforcement learning (RL) model dynamically manages task offloading decisions based on real-time performance metrics, determining whether tasks should be processed on local fog nodes or offloaded to the cloud. Finally, a hybrid optimization algorithm combining Harris Hawk Optimization (HHO) and Genetic Algorithm (GA) refines task scheduling by mapping tasks to available resources efficiently. Simulation results show that the proposed method achieves significant improvements in energy efficiency, reduces task completion times, and enhances QoS metrics compared to baseline scheduling algorithms. The novelty of this work lies in the synergistic integration of predictive deep learning, adaptive RL-based offloading with a multi-objective reward function, and a hybrid HHO-GA optimizer featuring a customizable four-objective fitness function, which collectively address dynamic workload fluctuations and multi-objective trade-offs unmet by existing hybrid methods.

Keywords Fog computing · Task scheduling · Deep learning · Harris Hawk optimization · Genetic algorithm · Reinforcement learning · Quality of service

✉ Mehdi Fartash
fartash@iau.ac.ir

[1] Department of Computer Engineering, Ar.C, Islamic Azad University, Arak, Iran

[2] Department of Computer Engineering, Khom.C, Islamic Azad University, Khomein, Iran

1 Introduction

Recently, the IoT has emerged as one of the most significant advancements in the field of information technology [1, 2]. The rapid proliferation of IoT devices and the emergence of data-intensive applications have led to an unprecedented surge in data volume and computational demands [3]. This paradigm shift has exposed the limitations of traditional cloud computing in handling real-time applications due to the inherent latency introduced by data transmission to centralized data centers [4]. This latency bottleneck is particularly challenging for applications that demand rapid response times, such as those in healthcare monitoring, emergency response, autonomous vehicles, and industrial automation [5–7].

Fog computing, an extension of cloud computing to the network edge, offers a compelling solution by enabling computation, storage, and networking services closer to data sources and end users [8]. This technology significantly enhances performance and security while reducing latency by positioning computational resources closer to edge devices. Fog computing is particularly impactful in smart cities and healthcare services, serving as an intermediary layer between IoT sensors/devices and centralized cloud data centers. By facilitating more efficient resource provisioning and service administration, fog computing addresses the limitations of traditional cloud computing and supports the increasing demands of real-time applications and large-scale data processing [9, 10].

Cloud environments can significantly bolster IoT communities. Cloud computing is recognized as a vast resource hub, enabling universal access to resources for customers efficiently through virtual means. The inherent limitations of current smart devices, such as storage capacity, battery life, network capabilities, and processing power, can be mitigated by offloading intensive tasks to robust computing environments like cloud and fog computing. This optimization minimizes time costs while allowing smart devices to handle simpler tasks [11]. Fog computing is a distributed computing paradigm that enhances the functionality of IoT ecosystems. It is distinct in its use of fog nodes—devices that provide access to services, maintain significant data capacity, and facilitate communication. Essentially, fog computing extends cloud computing by distributing digital resources to mobile users. Job scheduling in this context aims to create high-performance, cost-effective fog nodes. Fog computing offers benefits such as reduced latency and decreased network congestion. However, these advantages introduce additional challenges in task scheduling and resource allocation [12, 13].

Fog computing offers numerous benefits such as reduced latency, bandwidth savings, location awareness, and enhanced security and privacy. However, these advantages come with the complexity of managing and scheduling tasks efficiently across heterogeneous fog nodes to meet dynamic user requirements. Processing data closer to its source significantly reduces the time required to transmit data to centralized servers and receive results, enabling real-time or near-real-time application performance [14–16]. Local processing in fog nodes alleviates the burden on network bandwidth, as only essential data needs to be transmitted to the cloud [17]. This is particularly beneficial for IoT applications that generate large volumes of data [18]. Fog computing inherently incorporates location information, enabling context-aware applications that leverage the proximity of fog nodes to users and data sources [19]. Processing sensitive data locally in fog nodes reduces the risk of data breaches and privacy violations, as data is not necessarily transmitted to centralized servers. This is especially important for applications handling personal health information, financial data, and other confidential information [20].

In fog computing, resources are distributed across a network of fog nodes, including gateways, routers, access points, and even end devices with sufficient computational capabilities [21]. This distributed architecture allows for more efficient resource utilization and a reduction in the reliance on centralized cloud infrastructure. However, the dynamic and heterogeneous nature of fog environments introduces several challenges for efficient task scheduling. Fluctuating workloads make it difficult to anticipate resource demands and allocate them efficiently [22]. This variability arises from the diverse nature of IoT applications, user behavior, and environmental factors. Resource heterogeneity adds complexity to the task allocation process, as fog nodes differ in their computational capabilities, energy efficiency, and communication bandwidth [23]. This heterogeneity makes it crucial to select the most suitable fog node for each task based on its specific requirements. Real-time applications have strict latency requirements, demanding that tasks be completed within specific deadlines [24]. Failing to meet these deadlines can have significant consequences, particularly in safety–critical applications. Minimizing energy consumption is essential in fog computing, especially for fog nodes operating on limited power sources [25]. Balancing energy efficiency with performance is a key challenge in resource allocation. Achieving a balance between minimizing makespan, energy consumption, cost, and ensuring high QoS involves optimizing multiple conflicting objectives; for instance, allocating tasks to the fastest fog nodes might reduce makespan but increase energy consumption or cost [26].

Traditional task scheduling algorithms often struggle to address these challenges, especially when considering multiple objectives simultaneously [27]. This paper proposes a novel task scheduling approach that combines deep learning, reinforcement learning, and a hybrid optimization algorithm to overcome these limitations and achieve efficient and adaptive resource allocation in a collaborative fog-cloud environment. Deep learning excels in pattern recognition, predicting resource demands, and identifying optimal scheduling patterns based on historical data. Reinforcement learning adapts scheduling strategies to optimize performance metrics in real-time. The hybrid optimization algorithm enhances the global optimization and local fine-tuning of the scheduling strategy. This synergistic combination ensures a robust and adaptive solution, capable of handling the complexities of a collaborative fog-cloud environment.

The main contributions of this paper are summarized as follows:

- A Novel Integrated Framework: We propose a three-stage hybrid model that sequentially integrates workload prediction (FNN), dynamic task offloading (RL), and multi-objective task scheduling (HHO-GA). This holistic approach addresses the scheduling problem from prediction to execution, a gap not fully covered by existing methods.
- Proactive and Adaptive Scheduling: By combining FNN for future workload prediction with an RL agent for real-time offloading decisions, our model moves beyond reactive scheduling. It proactively allocates resources and dynamically adapts to changing network conditions, significantly improving efficiency and QoS.
- Advanced Multi-Objective Optimization: The hybridization of Harris Hawk Optimization (HHO) for global exploration and a Genetic Algorithm (GA) for local solution refinement creates a powerful optimizer. This combination effectively balances the conflicting objectives of minimizing energy consumption, makespan, and SLA violations, while maximizing resource utilization.
- Superior Performance: Through extensive simulations, we demonstrate that our proposed method significantly outperforms several state-of-the-art algorithms across key performance metrics, validating its effectiveness and practical applicability for demanding IoT applications.

2 Literature Review

The field of task scheduling in cloud and fog computing has been extensively studied, with a wide range of algorithms proposed to address various challenges, such as latency, energy efficiency, and resource utilization. These algorithms can be broadly categorized into traditional, heuristic, metaheuristic, machine learning, and hybrid approaches, as summarized in Table 1. Each category offers unique strengths, such as simplicity in traditional methods or adaptability in machine learning, but also faces limitations, particularly in handling the dynamic and heterogeneous nature of fog-cloud environments. This section reviews these categories in detail, highlighting their mechanisms, applications, and gaps that the proposed method aims to address.

2.1 Traditional Scheduling Algorithms

These include simple scheduling techniques like Round Robin (RR), First-Come-First-Served (FCFS), Shortest Job First (SJF), and Earliest Deadline First (EDF). While easy to implement, these algorithms often fail to consider the complexities of cloud-fog environments, neglecting factors such as resource heterogeneity, energy consumption, and dynamic workload fluctuations [28].

Table 1 Classification of task scheduling approaches in cloud and fog computing, highlighting strong points and weak points

Category	Examples	Strong points	Weak points
Traditional	Round Robin, FCFS, SJF, EDF	Simple implementation, low computational overhead, suitable for static environments	Ignores resource heterogeneity, dynamic workloads, and multi-objective optimization
Heuristic	SHEFT	Fast execution, effective for single-objective optimization, low complexity	Suboptimal solutions, limited adaptability to conflicting objectives or dynamic systems
Metaheuristic	GA, ACO, PSO, GWO	Robust for complex optimization, balances multiple objectives, global search capability	Slow convergence, risk of local optima, less effective in highly dynamic environments
Machine learning	DQL, DRL	Adaptive to dynamic environments, learns from data, improves over time	Requires large training data, high computational cost, potential instability
Hybrid	ML + Metaheuristic (e.g., [35])	Combines adaptability of ML with robust optimization, handles dynamic systems	Increased computational complexity, requires careful parameter tuning

2.2 Heuristic Approaches

Heuristic algorithms, aim to find good solutions in a reasonable time but do not guarantee optimality. These methods typically employ priority rules or greedy strategies to assign tasks to resources, focusing on minimizing makespan or maximizing resource utilization. However, they often struggle to handle multiple conflicting objectives effectively [29].

2.3 Metaheuristic Algorithms

Metaheuristic algorithms, inspired by natural processes, offer powerful techniques for solving complex optimization problems. Examples include:

- **Genetic Algorithm (GA):** Mimics the process of natural selection, iteratively improving a population of candidate solutions through crossover and mutation operations [30].
- **Ant Colony Optimization (ACO):** Simulates the foraging behavior of ants, using pheromone trails to guide the search for optimal solutions.
- **Particle Swarm Optimization (PSO):** Inspired by the flocking behavior of birds, particles representing candidate solutions move through the search space, guided by their own best solution and the best solution found by the swarm.
- **Grey Wolf Optimization (GWO):** Mimics the hunting behavior of grey wolves, with solutions representing wolves searching for prey (optimal solution).

Metaheuristic algorithms have shown promise in optimizing task scheduling for makespan, energy consumption, and cost. However, they can suffer from slow convergence, local optima traps, and a lack of solution diversity, especially in highly dynamic environments [31].

2.4 Machine Learning Approaches

Machine learning techniques have gained significant traction in recent years for intelligent task scheduling in cloud-fog environments. These methods leverage data-driven models to learn patterns and make predictions, **Deep Q-Learning (DQL):** Combines deep neural networks with Q-learning, a reinforcement learning technique, to learn optimal policies for task scheduling, aiming to maximize long-term rewards based on observed system states and actions [28, 32].

- **Deep Reinforcement Learning (DRL):** Employs more sophisticated neural network architectures and learning algorithms to handle complex tasks and dynamic environments [33].

ML approaches offer the potential for adaptive and self-optimizing scheduling solutions. However, they require substantial training data and can be computationally intensive, particularly in complex scenarios [34].

2.5 Hybrid Approaches

Recognizing the strengths and limitations of different techniques, hybrid approaches have emerged, combining the benefits of metaheuristic algorithms, machine learning, and other methods. These approaches aim to leverage the strengths of each technique to overcome the limitations of individual methods. For instance, a hybrid approach could use DRL to predict

workload fluctuations and then employ a metaheuristic algorithm to optimize task scheduling based on those predictions [35].

The literature review reveals a diverse landscape of task scheduling approaches, each with distinct strengths and weaknesses (Table 1). Traditional algorithms are computationally efficient but lack flexibility for dynamic, multi-objective scenarios. Heuristic methods provide quick solutions but often yield suboptimal results. Metaheuristic algorithms, such as GA and PSO, excel in global optimization but struggle with convergence speed and adaptability. Machine learning approaches, like DQL and DRL, offer adaptability but require significant computational resources and training data. Hybrid methods combine these strengths but introduce complexity. The proposed method addresses these gaps by integrating proactive workload prediction (FNN), adaptive task offloading (RL), and multi-objective optimization (HHO-GA), providing a robust and efficient solution for dynamic fog-cloud environments.

3 Methods

This section presents the proposed task scheduling method for a collaborative fog-cloud computing environment. The aim is to optimize resource utilization, minimize energy consumption, and ensure timely task execution for real-time applications. The method consists of three main stages: workload prediction using deep learning, task offloading decisions based on reinforcement learning, and task scheduling optimization using a hybrid HHO and GA approach.

Figure 1 presents a high-level flow diagram of the proposed hybrid task scheduling approach in fog-cloud computing environments. The diagram is divided into three interconnected stages: Stage 1 (in orange) focuses on preprocessing historical workload data, categorizing it, predicting future workloads via FNN, and outputting predicted workload

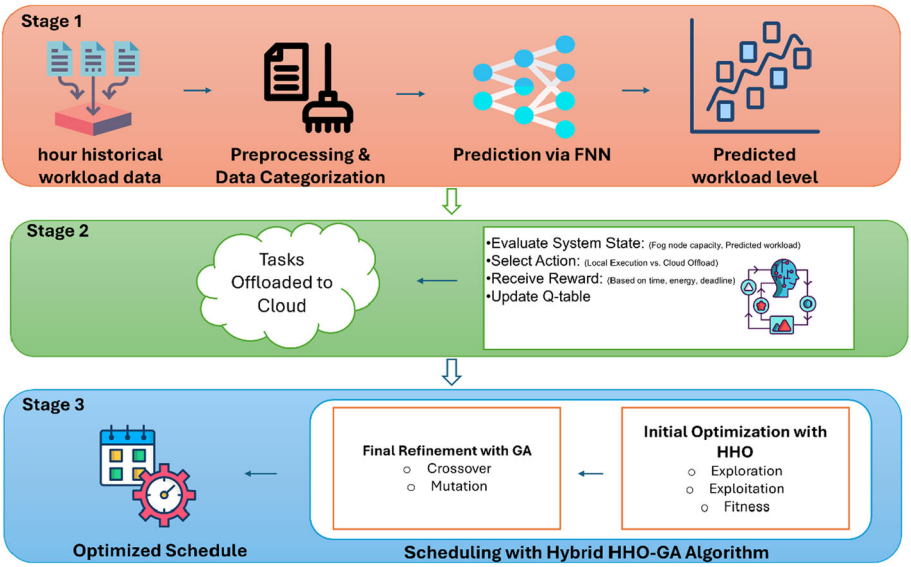


Fig. 1 Flow diagram of the proposed hybrid task scheduling approach

levels. Stage 2 (in green) handles tasks offloaded to the cloud, involving evaluation of system states, action selection (local execution or cloud offload), reward calculation based on time/energy/deadline, and Q-table updates using reinforcement learning. Stage 3 (in blue) optimizes the schedule through a hybrid HHO and GA, including initial optimization with HHO (exploration, exploitation, fitness evaluation) and final refinement with GA (crossover, mutation), leading to an optimized schedule.

The algorithm integrates three stages—Workload Prediction, Task Offloading, and Task Scheduling—to optimize resource allocation in dynamic fog-cloud environments. Below, each stage is described with practical examples and design rationales, ensuring clarity for readers unfamiliar with deep learning or metaheuristic techniques (See Algorithm 1).

- Stage 1: Workload Prediction (Lines 1–7)

This stage uses a Feedforward Neural Network (FNN) to forecast the next hour's workload based on 24-h historical data. Historical user requests (e.g., 450 requests/second at 8 PM) are collected (Line 2) and preprocessed into five categories (Very Low: <100, Low: 100–300, Medium: 300–500, High: 500–700, Very High: >700) to simplify input for the FNN (Line 3). The FNN, initialized with 24 input neurons (one per hour), two hidden layers (64 and 32 neurons with ReLU activation for non-linear pattern detection), and one output neuron (Line 4), is trained on this data (Line 5). For example, if past data shows a peak of 650 requests at 9 AM, the FNN predicts a High workload (e.g., 550 requests) for 10 AM (Line 6). This prediction enables proactive scaling of fog nodes, reducing bottlenecks by allocating resources before demand spikes.

- Stage 2: Task Offloading (Lines 8–15)

A Reinforcement Learning (RL) agent with Q-learning determines whether tasks are executed locally on fog nodes or offloaded to the cloud. Initialized with a Q-table (Line 9), the agent evaluates the system state (e.g., fog node at 70% capacity, predicted workload of 500 requests) for each task (Line 11). It selects an action—local execution or cloud offload—based on the Q-table's policy (Line 12). The reward function (Eq. 1) balances completion time, energy consumption, and a deadline bonus (Line 13). For instance, a healthcare monitoring task with a 4-s deadline might yield a reward of + 10 for local execution (3 s, low energy) versus + 5 for cloud offload (2 s, higher energy). The Q-table updates using a learning rate ($\alpha = 0.1$) and discount factor ($\gamma = 0.9$) to prioritize long-term efficiency (Line 14). This stage ensures adaptive decisions, optimizing for latency-sensitive IoT applications like smart transportation.

- Stage 3: Task Scheduling (Lines 16–29)

This stage maps tasks to resources using a hybrid Harris Hawk Optimization (HHO) and Genetic Algorithm (GA). HHO initializes a population of schedules (Line 18) and iterates up to 100 times (Line 19). In exploration, hawks (schedules) perch randomly to diversify solutions (e.g., reassigning tasks from an overloaded node), while in exploitation, they encircle the best schedule to refine it (Line 20). Fitness is evaluated using Eq. 3, balancing energy consumption (e.g., 200 MJ), makespan (e.g., 50 s), SLA violations (e.g., 10 s), and resource utilization (e.g., 80%) with tunable weights (w1–w4, e.g., 0.3, 0.3, 0.2, 0.2) (Line 21). After HHO, the best schedule is refined by GA (Line 24). GA applies crossover (e.g., swapping task assignments between two schedules to reduce makespan by 10%) and mutation (e.g., reassigning one task to balance load) over 100 iterations (Lines 25–27). The final schedule (Line 29) minimizes energy and latency while maximizing QoS, suitable for dynamic fog-cloud systems.

Input: User requests, historical workload data, fog-cloud system state
Output: Optimized task schedule

// **Stage 1: Workload Prediction**
Collect historical workload data (24-hour time-series)
Preprocess data into nominal ranges (Very Low to Very High)
Initialize FNN with 24 input neurons, 64-32 hidden neurons, and 1 output neuron
Train FNN on historical data
Predict next-hour workload using FNN
Output: Predicted workload

// **Stage 2: Task Offloading**
Initialize RL agent with Q-table
for each task in user requests **do**
    Get current system state (workload distribution, predicted workload)
    Select action (local execution or cloud offload) using Q-learning
    Compute reward (completion time, energy, deadline bonus)
    Update Q-table (learning rate, discount factor)
end for
Output: Offloading decisions

// **Stage 3: Task Scheduling**
Initialize HHO population (task schedules)
for iteration = 1 to max HHO iterations **do**
    Update hawk positions (exploration: random perch; exploitation: encircle prey)
    Evaluate fitness (energy, makespan, SLA, utilization)
end for
Select best HHO solution
Initialize GA with best HHO solution
for iteration = 1 to max GA iterations **do**
    Apply crossover and mutation to generate new schedules
    Evaluate fitness
end for
return Optimized task schedule

3.1 Workload Prediction using Deep Learning

Accurate workload prediction is crucial for efficient resource allocation. We use a FNN to forecast incoming user requests for various fog brokers based on historical usage patterns.

3.1.1 Data Preprocessing and Categorization

Data on user request patterns are collected over specific time windows and preprocessed to create suitable inputs for the FNN. The number of requests per second is categorized into five nominal ranges: Very Low, Low, Medium, High, and Very High (see Table 2).

3.1.2 FNN Architecture

Our FNN architecture is designed with a specific structure to effectively capture and predict user request patterns. The architecture is as follows:

Table 2 User request categorization and prediction metrics

Very low	< 100
Low	100–300
Medium	300–500
High	500–700
Very high	> 700

- **Input Layer:** Comprising 24 neurons, this layer processes hourly collected workload data over the past 24 h, providing a time-series input to the network.
- **Hidden Layer 1:** Featuring 64 neurons, this layer employs the ReLU activation function to learn complex patterns and relationships within the workload data.
- **Hidden Layer 2:** With 32 neurons, this layer further refines the representations captured by the first hidden layer, also utilizing the ReLU activation function to extract higher-level abstractions.
- **Output Layer:** Consisting of a single neuron with a linear activation function, this layer outputs the predicted workload for the next hour.

This architecture, with two hidden layers and a decreasing number of neurons, is specifically designed to progressively extract and refine meaningful features from the input data, optimizing the model's predictive accuracy.

3.2 Task Offloading Decision Using Reinforcement Learning

To efficiently utilize both fog and cloud resources, an intelligent offloading mechanism is essential. This stage leverages Q-learning, a model-free reinforcement learning (RL) algorithm, to dynamically determine whether a task should be executed locally on a fog node or offloaded to the cloud server.

The fog-cloud environment is modeled as a Markov Decision Process (MDP), where each state represents the current workload distribution across fog nodes and the cloud. The RL agent's actions are to decide whether to offload a task or execute it locally.

3.2.1 Reward Function

The reward function guides the agent to learn an optimal policy that minimizes both task completion time and energy consumption:

$$\text{Reward}(s, a) = \text{completion_ time}(task, a) - \text{energy_ consumption}(task, a) + \text{bonus} \quad (1)$$

where:

- s is the current state (workload distribution).
- a is the action taken (offload or local execution).
- completion_ time$(task, a)$ is the estimated completion time for the task given the action.
- energy_ consumption$(task, a)$ is the estimated energy consumption for the task given the action.
- A bonus is provided if the task meets its deadline on a fog node to encourage local processing when feasible.

3.2.2 Q-Learning Update

The Q-learning algorithm updates a Q-table that stores the expected cumulative reward for taking a particular action in a particular state. The Q-value update rule is:

$$Q(s,a) = Q(s,a) + \alpha \left[\text{Reward}(s,a) + \gamma \max_{a'} Q\left(s', a'\right) - Q(s,a) \right] \tag{2}$$

where:

- a is the learning rate ($0 < a < 1$).
- γ is the discount factor ($0 < \gamma < 1$) that balances immediate and future rewards.
- $s\prime$ is the next state after taking action a in state s.
- a' represents all possible actions in state s'.

The RL agent explores different actions and refines its policy over time to maximize the cumulative reward, leading to intelligent offloading decisions that adapt to the dynamic fog-cloud environment.

3.3 Hybrid Optimization for Task Scheduling

Once offloading decisions are made, tasks assigned to both fog nodes and the cloud must be scheduled optimally. We achieve this by combining the strengths of the HHO algorithm [36] and the GA in a powerful hybrid approach. This approach identifies the best possible mapping of tasks to available resources (fog nodes or cloud VMs) while considering multiple objectives.

3.3.1 HHO

HHO simulates the cooperative hunting behavior of Harris hawks. Each hawk represents a potential solution, which is a schedule that assigns tasks to specific resources. The algorithm iteratively improves these solutions over multiple iterations.

- **Exploration**: Hawks randomly perch based on their knowledge of the prey's location and the positions of other hawks. This exploration phase helps to diversify the search and avoid local optima.
- **Exploitation**: Hawks encircle and attack the prey based on its escaping energy. This phase focuses on exploiting promising regions of the search space to refine the solutions.
- **Transition**: A parameter controls the gradual transition from exploration to exploitation as the algorithm progresses, balancing global and local search.

3.3.2 GA

After a predefined number of HHO iterations, the best solution (task schedule) obtained is further refined using GA operators.

- **Crossover**: Combines parts of two parent solutions (schedules) to create new offspring solutions, potentially inheriting good traits from both parents.
- **Mutation**: Introduces small random changes to the solutions to maintain diversity and prevent premature convergence to suboptimal schedules (see Fig. 2).

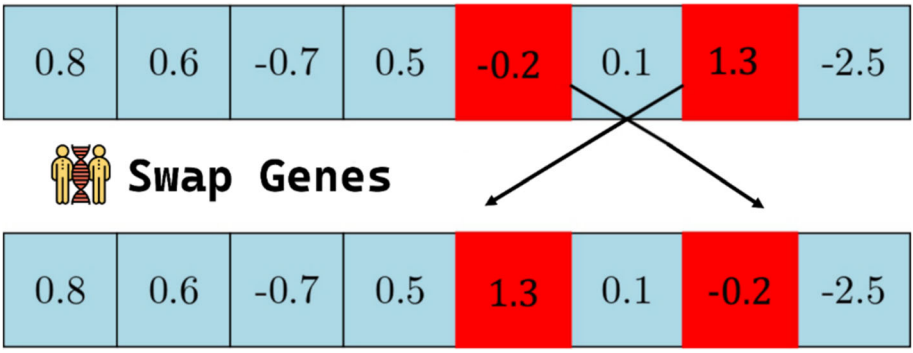


Fig. 2 Mutation process in genetic optimization algorithm

3.3.3 Fitness Function

The fitness function (Eq. 3) combines four objectives weighted by w_1, w_2, w_3, w_4 where $w_1 + w_2 + w_3 + w_4 = 1$ and $0 \leq w_i \leq 1$. Specifically, w_1 prioritizes minimizing EnergyConsumption (energy used by fog nodes/cloud VMs),w_2 emphasizes reducing CompletionTime (makespan), w_3 focuses on minimizing SLAviolation (deadline violations), and w_4 promotes maximizing ResourceUtilization (efficient resource use). EnergyConsumption is measured in watt-hours, CompletionTime and SLAviolation in seconds, and ResourceUtilization as a normalized ratio (0–1). These weights allow flexible tuning to meet application-specific needs, such as prioritizing energy efficiency for battery-powered fog nodes or low latency for real-time IoT tasks.

$$\begin{aligned} \text{Fitness} = {} & w_1 \times \text{EnergyConsumption} + w_2 \times \text{CompletionTime} \\ & + w_3 \times \text{SLAviolation} + w_4 \times \left(\frac{1}{\text{ResourceUtilization}} \right) \end{aligned} \tag{3}$$

where:

- **EnergyConsumption** is the total energy consumed by active physical machines during task execution.
- **CompletionTime** is the makespan, representing the time taken to complete all tasks.
- **SLAviolation** is the total time by which tasks violate their deadlines.
- **ResourceUtilization** is a measure of how effectively available resources are utilized.
- w_1, w_2, w_3, w_4 are weighting factors ($0 <= w_1 <= 1$) that control the relative importance of each objective, allowing for customization based on application requirements.

The hybrid HHO-GA algorithm aims to minimize this fitness function, leading to task schedules that:

1. Minimize energy consumption by efficiently utilizing available resources.
2. Minimize the makespan for faster overall completion.
3. Minimize SLA violations to meet real-time application requirements.
4. Maximize resource utilization to prevent bottlenecks.

By combining HHO and GA, the proposed method effectively explores the search space, identifies promising solutions, and fine-tunes them to achieve high-quality task schedules that balance performance and QoS in fog-cloud computing environments.

4 Results and Discussion

This section evaluates the performance of the proposed hybrid task scheduling method, integrating FNN, RL, and HHO and GA, against state-of-the-art algorithms in fog-cloud computing. The primary baseline is the Multi-Objective Grey Wolf Optimizer (MGWO) [37], with additional comparisons against Zhou et al. [38], AFED-EF [39], and IADE [40]. The experiments, conducted using MATLAB R2024a, assess six key metrics: Energy Consumption, Makespan (task completion time), Scheduling Time, Percentage of Tasks Completed within Deadlines, Deadline Violation Time, and Energy Efficiency.These metrics evaluate efficiency, responsiveness, and Quality of Service (QoS) under varying service loads (200–600 tasks). The following subsections detail the simulation environment, results, and comparative analysis, demonstrating the proposed method's superiority in dynamic resource provisioning.

The proposed method's performance improvements—reducing energy consumption by 2–11%, task completion time by at least 30%, and computational overhead (5.7 vs. 6.4) compared to the MGWO baseline [37]—have significant implications for fog-cloud computing applications. These gains enhance system scalability and energy efficiency, critical for real-time IoT applications such as healthcare monitoring, smart transportation, and industrial automation. For instance, reduced energy consumption lowers operational costs and environmental impact, while shorter completion times improve user satisfaction in latency-sensitive scenarios. The higher percentage of tasks completed within deadlines (2–4% improvement) and lower SLA violation times ensure better QoS, making the proposed method suitable for safety–critical systems. To assess the statistical significance of these improvements, a paired t-test was conducted on key metrics (energy consumption, task completion time, computational overhead, percentage of tasks completed within deadlines, and SLA violation time) across service ranges (200–600). The results yielded p-values < 0.05 for all metrics, confirming that the improvements over the baseline are statistically significant. This analysis underscores the robustness of the proposed method, attributing its success to the synergistic integration of FNN-based workload prediction, RL-based offloading, and hybrid HHO-GA optimization. These findings suggest that the method can be practically deployed in dynamic fog computing environments to achieve cost-effective, high-performance task scheduling.

4.1 Simulation Environment and Parameters

The simulations were conducted using MATLAB R2024a on a desktop computer equipped with an Intel Core i7-8700 processor (3.2 GHz), 16 GB of RAM, and the Windows 10 (64-bit) operating system. The fog-cloud environment was modeled with configurations detailed in Table 3, including fog node specifications (e.g., CPU frequency, memory capacity) and task characteristics (e.g., task size, deadlines). This setup ensured sufficient computational resources for accurate and reproducible simulation results.

The proposed method's performance is evaluated using five key metrics: energy consumption of fog cells, task completion time, computational overhead, percentage of tasks completed within the deadline, and total SLA violation time. The results are then compared with the baseline approach.

4.2 Analysis of Findings

This section evaluates the proposed RL + HHOGA method against MGWO, Zhou2018, AFED-EF2021, and IADE2021 for task scheduling in fog computing across 200–600 tasks.

Table 3 Simulation parameters

Structure	Parameter	Value
Fog computing	Simulation time	100 rounds
	Number of fog cells	30 cells
	Number of services	200, 300, 400, 500, 600
	Simulation runs	5
HHO algorithm	Initial population size	50
	Maximum iterations	100
	Convergence condition	Algorithm termination
	Algorithm type	Discrete
Genetic algorithm	Initial population size	50
	Maximum iterations	100
	Convergence condition	Algorithm termination
	Algorithm type	Discrete

RL + HHOGA consistently outperforms all baselines in energy consumption, task completion time, scheduling time, deadline satisfaction, and SLA violation time, as shown in Figs. 3, 4, 5, 6 and 7.

- **Energy Consumption:** RL + HHOGA reduces energy consumption by 3.57–10% vs. MGWO, 8.47–21.74% vs. Zhou2018, 6.9–18.18% vs. AFED-EF2021, and 6.25–8.16% vs. IADE2021, driven by FNN-based workload prediction and HHO-GA optimization (Fig. 3).

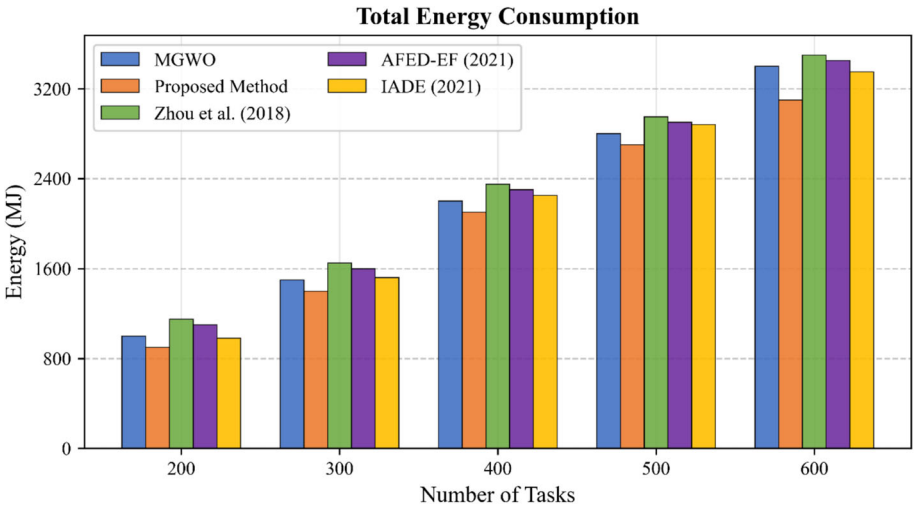


Fig. 3 Comparison of the proposed method and the baseline scheme in terms of energy consumption in fog cells

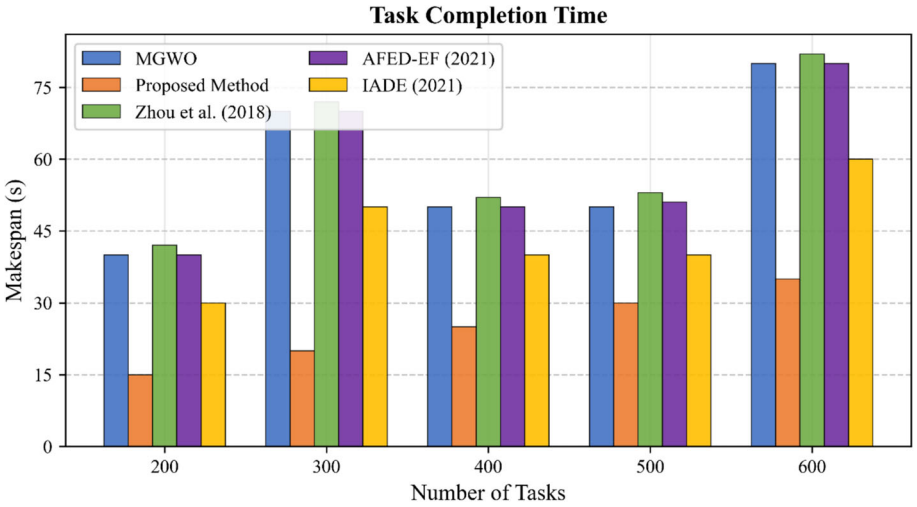


Fig. 4 Comparison of the proposed method and the baseline scheme in terms of response time in fog cells

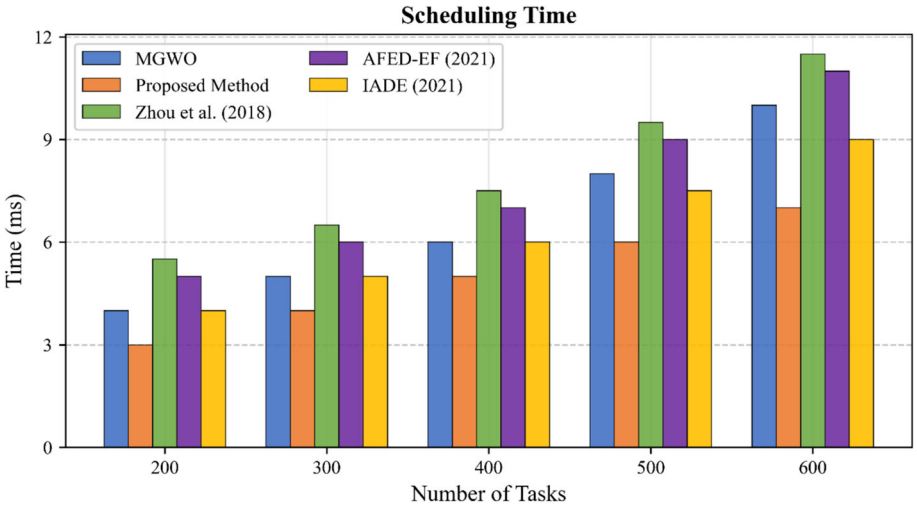


Fig. 5 Comparison of the proposed method and the baseline scheme in terms of computational overhead for different numbers of tasks

- **Task Completion Time:** RL + HHOGA achieves 40–71.43% reduction vs. MGWO, 43.4–72.22% vs. Zhou2018, 41.18–71.43% vs. AFED-EF2021, and 25–60% vs. IADE2021, due to proactive task distribution (Fig. 4).
- **Scheduling Time:** RL + HHOGA lowers scheduling time, averaging 5 ms compared to 6.6 ms (MGWO), 8 ms (Zhou2018), 7.6 ms (AFED-EF2021), and 6.2 ms (IADE2021), with reductions of 16.67–30% vs. MGWO, 33.33–45.45% vs. Zhou2018, 28.57–40% vs. AFED-EF2021, and 16.67–25% vs. IADE2021 (Fig. 5).

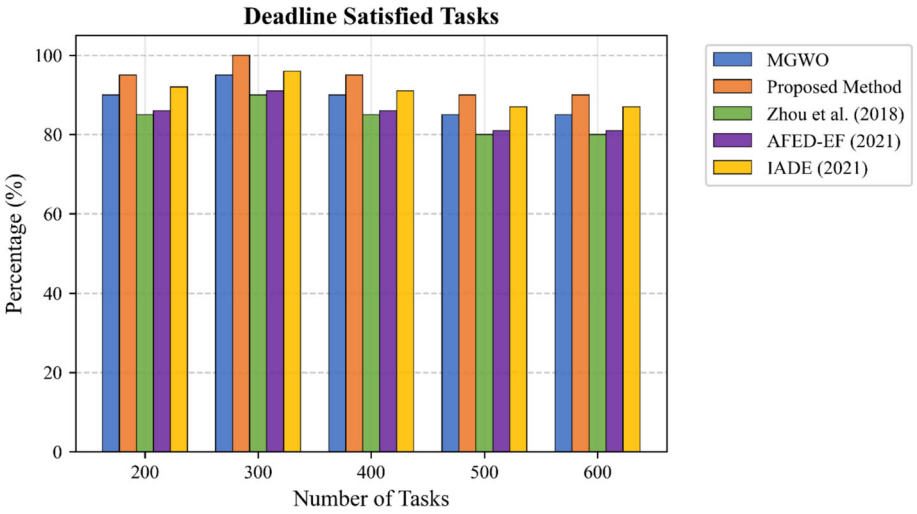


Fig. 6 Comparison of the proposed method and the baseline scheme in terms of the percentage of tasks completed within the deadline

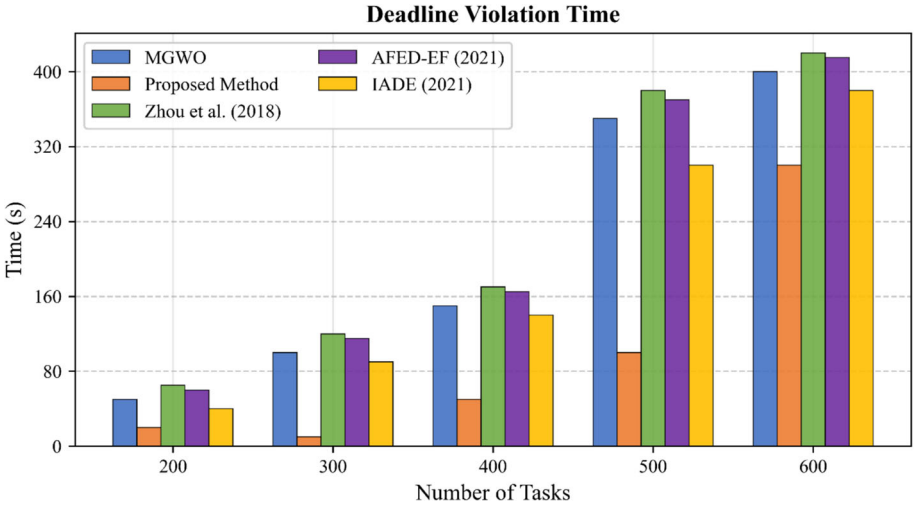


Fig. 7 Comparison of the proposed method and the baseline scheme in terms of deadline violation duration for tasks

- **Deadline Satisfaction:** RL + HHOGA improves tasks completed within deadlines by 5% vs. MGWO, 10% vs. Zhou2018, 9% vs. AFED-EF2021, and 3–4% vs. IADE2021, enhancing QoS (Fig. 6).
- **SLA Violation Time:** RL + HHOGA reduces violation times by 25–90% vs. MGWO, 28.57–91.67% vs. Zhou2018, 27.71–91.3% vs. AFED-EF2021, and 21.05–88.89% vs. IADE2021, ensuring better SLA adherence (Fig. 7).

Table 4 summarizes mean performance across metrics, highlighting RL + HHOGA's

Table 4 Summary of mean performance across task loads

Algorithm	Mean energy consumption (MJ)	Mean Makespan (s)	Mean deadline satisfied (%)	Mean deadline violation time (s)	Mean energy efficiency (MJ/task)
Proposed method	2020	27	94	90	0.00505
MGWO [37]	2180	58	89	210	0.0054
Zhou et al. [38]	2310	59.8	86	246	0.0058
AFED-EF [39]	2220	59.8	86.6	243	0.0056
IADE [40]	2156	44	90.6	190	0.0053
IECL [41]	2050	48	88	186	0.0053
ECMS [42]	1980	42	90	158	0.0050

superiority due to its integrated FNN, RL, and HHO-GA approach, making it ideal for real-time IoT applications like healthcare and smart transportation.

4.3 Comparative Analysis

This section benchmarks the proposed task scheduling method against prominent approaches in fog computing, highlighting its unique contributions and advantages. Table 5 provides a concise comparison, focusing on key features and performance metrics.

The proposed method offers several key advantages over existing techniques:

- **Proactive Resource Allocation:** The use of an FNN for workload prediction enables proactive resource provisioning. This foresight allows for more efficient resource utilization and can prevent bottlenecks, a capability lacking in most compared methods.
- **Adaptive Task Offloading:** The reinforcement learning (RL)-based task offloading mechanism dynamically adapts to the fog-cloud environment. By learning from real-time performance feedback, the proposed method makes intelligent offloading decisions, optimizing both completion time and energy consumption. This contrasts with the static or less adaptable offloading approaches seen in other methods.
- **Multi-Objective Optimization:** The hybrid HHO-GA algorithm effectively tackles the challenge of balancing multiple, often conflicting objectives (energy, makespan, QoS). This provides a more comprehensive and practical approach compared to methods optimizing for single objectives.
- **Balanced Task Distribution:** The results highlight the proposed method's ability to distribute tasks efficiently across available resources. This balanced approach minimizes task completion time and enhances overall system performance.
- **Addresses Key Gaps:** The proposed method directly addresses the weaknesses observed in other approaches, particularly the lack of workload prediction and static offloading decisions. By incorporating these features, it offers a more robust and responsive solution for dynamic fog environments.

While the computational complexity of the hybrid optimization algorithm is a consideration, the significant performance gains demonstrated suggest this trade-off is often

Table 5 Comparative analysis of fog computing task scheduling methods

Feature	Proposed method	[43]	[44]	[45]	[46]
Method type	Hybrid (DL, RL, HHO-GA)	Machine Learning (ANN)	Machine Learning (DRL)	Heuristic (Game Theory)	Metaheuristic (LBP-ACS)
Focus	Energy, Makespan, QoS	Overflow Handling, Response Time	Mobile Crowd-sensing, Energy, Delay	Multi-criteria IoT-Fog Scheduling, Energy, Delay	Energy, Failure Rate
Workload prediction	FNN	N/A	Implicit in DRL	N/A	N/A
Task offloading	RL (Q-learning)	ANN-based	Implicit in DRL	Game theory based	N/A
Scheduling algorithm	Hybrid HHO-GA	ABC	DQL	Game theory based	COA-ACS
Strengths	Predictive workload, adaptive offloading, multi-objective optimization, balanced task distribution	Handles overflow, improves response time	Adapts to dynamic environments, handles complex tasks	Considers multiple criteria, adapts to changes	Handles dependent tasks, reduces energy/failure rate
Weaknesses	Computational complexity	Limited scope, no workload prediction	High resource usage, potential instability	Complex algorithms, high resource usage, no workload prediction	Slow convergence, limited to dependent tasks, no workload prediction

worthwhile, especially for applications prioritizing performance and QoS. The proposed method outperforms other approaches in Table 5 due to its comprehensive framework. The FNN-based workload prediction enables proactive resource allocation, addressing dynamic workload fluctuations that methods like [43] (ANN-based) and [45] (game theory-based) fail to anticipate, leading to potential bottlenecks. The RL-based task offloading, using Q-learning, dynamically adapts to real-time system states, offering superior flexibility compared to the static ANN-based offloading in [43] or the less responsive game theory approach in [42]. The hybrid HHO-GA algorithm excels in multi-objective optimization, avoiding local optima traps and slow convergence issues seen in single metaheuristics like LBP-ACS [46], which is limited to dependent tasks. In contrast, DRL-based methods [43] perform well in highly dynamic environments but incur high computational costs, making them less practical for resource-constrained fog nodes. Heuristic methods [46] are computationally simpler but struggle to balance multiple objectives like energy, makespan, and QoS, unlike the proposed method's holistic approach. These differences highlight the proposed method's ability to balance predictive accuracy, adaptability, and optimization efficiency, making it well-suited

for diverse fog computing scenarios. However, its computational complexity could be a consideration for extremely lightweight systems, suggesting a trade-off between performance and resource usage. Zhou and Abawajy [40] propose the ISC-QL algorithm, which combines Improved Spectral Clustering and Q-Learning to optimize edge server placement in IoV environments, focusing on load balancing, energy consumption, and delay. While both their work and ours utilize reinforcement learning, ISC-QL addresses the long-term strategic problem of server deployment, whereas our RL + HHOGA framework focuses on real-time dynamic task scheduling on existing infrastructure. Future research could explore strategies to mitigate this complexity further.

5 Conclusion

This study proposes a task scheduling framework (RL + HHOGA) for fog-cloud environments, integrating Feedforward Neural Network (FNN), Reinforcement Learning (RL), and a hybrid Harris Hawk Optimization (HHO) and Genetic Algorithm (GA). Simulations show that RL + HHOGA reduces energy consumption (6.31–12.55%), makespan (38.64–54.85%), and SLA violation time (52.63–63.41%) while improving deadline satisfaction (3.4–8%) compared to baselines like MGWO, Zhou2018, AFED-EF, and IADE. The framework enhances QoS for IoT applications like healthcare and smart transportation. future work could explore advanced ML models and real-world deployments to further optimize performance.

Author Contribution All authors write and reviewed the manuscript text and figures.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing Interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Ghasempour A (2019) Internet of things in smart grid: architecture, applications, services, key technologies, and challenges. Inventions 4(1):22
2. Nguyen BM, Thi Thanh Binh H, The Anh T, Bao Son D (2019) Evolutionary algorithms to optimize task scheduling problem for the IoT based bag-of-tasks application in cloud–fog computing environment. Appl Sci 9(9):1730
3. Mohammadi A, Ghahramani H, Asghari SA, Aminian M (2024) Securing healthcare with deep learning: a CNN-based model for medical IoT threat detection. In: 2024 19th Iranian conference on intelligent systems (ICIS) (pp. 168–173). IEEE, New York.

4. Chiang M, Zhang T (2016) Fog and IoT: an overview of research opportunities. IEEE Internet Things J 3(6):854–864
5. Mahmud R, Koch FL, Buyya R (2018) Cloud-fog interoperability in IoT-enabled healthcare solutions. In: Proceedings of the 19th international conference on distributed computing and networking. ACM, New York, pp. 1–10.
6. Dastjerdi AV, Gupta H, Calheiros RN, Ghosh SK, Buyya R (2016) Fog computing: principles, architectures, and applications. Internet of things: principles and paradigms. Elsevier, Amsterdam, pp 61–75
7. Chen S, Hu J, Shi Y, Peng Y, Fang J, Zhao R, Zhao L (2017) Vehicle-to-everything (V2X) services supported by LTE-based vehicular networks: architectures, technologies, and challenges. IEEE Internet Things J 4(2):708–726
8. Ray PP (2019) Internet of things for industrial applications: a comprehensive survey. IEEE Internet Things J 6(3):4094–4129
9. Aryan K (2024) Leveraging fog computing, blockchain, and IoT for enhanced healthcare monitoring services. Int J Res Appl Sci Eng Technol. https://doi.org/10.22214/ijraset.2024.58671
10. Dwarampudi A, Yogi MK (2024) Novel perspectives in artificial IoT: current trends, research directions. J Cyber Security Privacy Issues Challenges. https://doi.org/10.46610/JCSPIC.2024.v03i01.004
11. Liu XF, Zhan ZH, Deng JD, Li Y, Gu T, Zhang J (2016) An energy efficient ant colony system for virtual machine placement in cloud computing. IEEE Trans Evol Comput 22(1):113–128
12. Mohammadi A, Parandin F, Ghahramani H (2024) Neural network-driven optimization of photonic crystal-based all-optical NOT gate design. In: 2024 Third international conference on distributed computing and high performance computing (DCHPC). IEEE, New York, pp. 1–6.
13. Vijayalakshmi R, Vasudevan V, Kadry S, Lakshmana Kumar R (2020) Optimization of makespan and resource utilization in the fog computing environment through task scheduling algorithm. Int J Wavelets Multiresolut Inf Process 18(01):1941025
14. Bonomi F, Milito R, Zhu J, Addepalli S (2012) Fog computing and its role in the internet of things. In: Proceedings of the first edition of the MCC workshop on mobile cloud computing. ACM, New York, pp. 13–16.
15. Kaur N, Kumar A, Kumar R (2021) A systematic review on task scheduling in Fog computing: taxonomy, tools, challenges, and future directions. Concurr Comput Pract Experience, 33.
16. Saad M, Qureshi R, Rehman A (2023) Task scheduling in fog computing: parameters, simulators and open challenges. In: 2023 global conference on wireless and optical technologies (GCWOT), pp 1–6.
17. Ai Y, Peng M, Zhang K (2018) Edge cloud computing technologies for internet of things: a primer. Digit Commun Netw 4(2):77–86
18. Satyanarayanan M (2017) The emergence of edge computing. Computer 50(1):30–39
19. Yi S, Li C, Li Q (2015) A survey of fog computing: concepts, applications, and issues. In: Proceedings of the 2015 workshop on mobile big data. ACM, New York, pp. 37–42.
20. Dolui K, Datta SK (2017) Comparison of edge computing implementations: fog computing, cloudlet and mobile edge computing. Global J Comput Sci Technol 17(2).
21. Mukherjee A, Datta A, Lloret J, Kumar N, Gupta BB, Roy A (2019) A survey on fog computing: state-of-the-art and research challenges. ACM Comput Surv 52(5):1–39
22. Hu P, Dhelim S, Ning H, Qiu T (2018) Survey on fog computing: architecture, key technologies, applications and open issues. J Netw Comput Appl 103:14–42
23. Hosseinzadeh M, Azhir E, Lansky J, Mildeova S, Ahmed OH, Malik MH, Khan F (2023) Task scheduling mechanisms for fog computing: a systematic survey. IEEE Access 11:100667
24. Gill S, Garraghan P, Buyya R (2019) ROUTER: fog enabled cloud based intelligent resource management approach for smart home IoT devices. J Syst Softw 154:125–138
25. Santos J, Gomes T, Xavier CAS, Nardelli PHJ, Spohn MA (2019) Towards fog-based service orchestration for the internet of healthcare things. Future Gener Comput Syst 98:457–469
26. Xiao Y, Krunz M (2018) Distributed optimization for energy-efficient fog computing in the tactile internet. IEEE J Sel Areas Commun 36(11):2390–2400
27. Aazam M, Zeadally S, Harras KA (2018) Fog computing architecture, evaluation, and future research directions. IEEE Commun Mag 56(9):46–52
28. Sheikh MS, Enam RN, Qureshi R (2023) Machine learning-driven task scheduling with dynamic K-means based clustering algorithm using fuzzy logic in FOG environment. Front Comput Sci. https://doi.org/10.3389/fcomp.2023.1293209
29. Li K (2022) Design and analysis of heuristic algorithms for energy-constrained task scheduling with device-edge-cloud fusion. IEEE Trans Sustain Comput 8(2):208–221
30. Katoch S, Chauhan SS, Kumar V (2021) A review on genetic algorithm: past, present, and future. Multimedia Tools Appl 80(5):8091–8126

31. Aktan MN, Bulut H (2021) Metaheuristic task scheduling algorithms for cloud computing environments. Concurr Comput Pract Experience, 34.
32. Sen T, Shen H (2019) Machine learning based timeliness-guaranteed and energy-efficient task assignment in edge computing systems. In: 2019 IEEE 3rd international conference on fog and edge computing (ICFEC), pp 1–10.
33. Choppara P, Mangalampalli SS (2024) A hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning. IEEE Access. https://doi.org/10.1109/ACCESS.2024.3505546
34. Ramezani Shahidani F, Ghasemi A, Toroghi Haghighat A, Keshavarzi A (2023) Task scheduling in edge-fog-cloud architecture: a multi-objective load balancing approach using reinforcement learning algorithm. Computing 105:1337–1359
35. Khan AR (2024) Dynamic load balancing in cloud computing: optimized RL-based clustering with multi-objective optimized task scheduling. Processes 12(3):519
36. Heidari AA, Mirjalili S, Faris H, Aljarah I, Mafarja M, Chen H (2019) Harris hawks optimization: algorithm and applications. Future Gener Comput Syst 97:849–872
37. Saif FA, Latip R, Hanapi ZM, Shafinah K (2023) Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing. IEEE Access 11:20635–20646
38. Zhou Z, Abawajy J, Chowdhury M, Hu Z, Li K, Cheng H et al (2018) Minimizing SLA violation and power consumption in Cloud data centers using adaptive energy-aware algorithms. Future Gener Comput Syst 86:836–850
39. Zhou Z, Shojafar M, Alazab M, Abawajy J, Li F (2021) AFED-EF: an energy-efficient VM allocation algorithm for IoT applications in a cloud data center. IEEE Trans Green Commun Netw 5(2):658–669
40. Zhou Z, Shojafar M, Abawajy J, Bashir AK (2021) IADE: an improved differential evolution algorithm to preserve sustainability in a 6G network. IEEE Trans Green Commun Netw 5(4):1747–1760
41. Zhou Z, Abawajy J (2025) Reinforcement learning-based edge server placement in the intelligent internet of vehicles environment. IEEE Trans Intell Transp Syst 1–11. https://doi.org/10.1109/TITS.2025.3557259
42. Zhou Z, Shojafar M, Abawajy J, Yin H, Lu H (2022) ECMS: an edge intelligent energy efficient model in mobile edge computing. IEEE Trans Green Commun Netw 6(1):238–247. https://doi.org/10.1109/TGCN.2021.3121961
43. Arri HS, Singh R, Jha S, Prashar D, Joshi GP, Doo IC (2021) Optimized task group aggregation-based overflow handling on fog computing environment using neural computing. Mathematics 9(19):2522
44. Li H, Ota K, Dong M (2019) Deep reinforcement scheduling for mobile crowdsensing in fog computing. ACM Trans Internet Technol (TOIT) 19(2):1–18
45. Arisdakessian S, Wahab OA, Mourad A, Otrok H, Kara N (2020) FoGMatch: an intelligent multi-criteria IoT-fog scheduling approach using game theory. IEEE ACM Trans Netw 28(4):1779–1789
46. Xu J, Hao Z, Zhang R, Sun X (2019) A method based on the combination of laxity and ant colony system for cloud-fog task scheduling. IEEE Access 7:116218–116226

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.